

Sri Lanka Institute of Information Technology



TryHackMe Room Report: Smuggler's Paradise

Assignment 1

Year 2 Semester 2

Batch: Weekend

Submitted by: - IT23363120

Web Security - IE2062

B.Sc. (Hons) in Information Technology Specializing in Cyber Security

1. Room Overview

Title: *Smuggler's Paradise – Mastering HTTP Request Smuggling Attacks*

Objective Summary:

In *Smuggler's Paradise*, learners are introduced to one of the most misunderstood and underutilized vulnerabilities — **HTTP Request Smuggling**. This room takes them through an engaging and hands-on journey, where they learn to exploit header misinterpretations between a proxy and a backend server. The final goal? Smuggle a hidden request to access a protected admin panel and uncover a secret flag.

Key Concepts & Vulnerabilities Covered:

- HTTP protocol structure and behavior
- Misinterpretations caused by Content-Length vs. Transfer-Encoding
- HTTP Request Smuggling attack vectors
- Real-world impact: authentication bypass, cache poisoning, and data theft
- Manual crafting and sending of raw HTTP requests using low-level tools like netcat

2. Learning Objectives

By the end of this room, users will be able to:

- Understand the anatomy of HTTP requests and headers.
- Identify how reverse proxies and backend servers process requests differently.
- Explain and demonstrate how request smuggling attacks work.
- Deploy a vulnerable Node.js server both locally and using the TryHackMe AttackBox.
- Use tools such as curl, nmap, and netcat to perform enumeration and exploit delivery.
- Craft smuggled requests manually to exploit header discrepancies and access protected resources.

3. Room Structure

Task 1: Room Overview

- This task gives users a thematic introduction to the room, describing the context of HTTP Request Smuggling and its real-world significance.
- Sets the tone and explains what learners will achieve by the end of the room.
- No question included here

Task 2: Prerequisites

- Users are guided to prepare their lab environment using either:
 - The TryHackMe AttackBox (recommended for convenience), or
 - Their own personal VM (instructions provided).
- A vulnerable server.js is supplied.
- Users are instructed to:
 1. Copy and paste the Node.js server code.
 2. Save it as server.js in their local VM or AttackBox.
 3. Install dependencies (e.g., npm install express).
 4. Start the server with node server.js.
 5. Confirm it is working by visiting /admin.
- This task ensures users understand and replicate the environment where the vulnerability exists.
- No question included here.

Task 3: What is HTTP?

- Teaches the fundamentals of the HTTP protocol:
 - Request/response structure
 - Headers and methods (GET, POST, etc.)
 - Stateless communication between client and server
- Uses analogies like sending a letter to explain the flow of information.

MCQ included to test understanding:

- What is HTTP primarily used for?
 - A) Encrypting files on a server
 - B) Sending commands to the operating system
 - C) Communication between browsers and servers

Answer: C

Task 4: What is a Proxy and Backend Server?

- Introduces the concept of proxy servers and backend servers:
 - Explains reverse proxies like NGINX or Apache
 - Discusses how proxies handle requests and forward them
 - Backend's role in interpreting those forwarded requests
- Uses real-world analogies (e.g., front desk receptionist and manager in a hotel).
- MCQ included:

Q1. What is the role of a proxy server?

- A) To directly respond to all client requests
- B) To forward requests to the backend server
- C) To block all unauthorized traffic

Answer: B

Task 5: What is HTTP Request Smuggling?

- Breaks down the vulnerability:
 - Describes how mismatched parsing by proxy vs. backend can be exploited
 - Uses nested envelope/mail analogy to make the concept intuitive
 - Explains real-world consequences (e.g., cache poisoning, bypassing auth)
- Demonstrates how attackers use Content-Length and Transfer-Encoding conflicts.
- MCQ included:

Q1. What headers are typically manipulated in HTTP Request Smuggling attacks?

- A) Accept and User-Agent
- B) Content-Length and Transfer-Encoding
- C) Host and Referer

Answer: B

Q2. What is HTTP Request Smuggling?

- A) A type of XSS attack
- B) A vulnerability exploiting header discrepancies
- C) A brute-force attack on password fields

Answer: B

Task 6: Reconnaissance

- Users learn to:
 - Scan and fingerprint the target server using tools like nmap, curl, and whatweb
 - Identify if a proxy is involved
 - Look at response headers and server behaviors
- Encourages habit of observing request-response pairs for abnormalities.
- MCQ included:

Q1. Which tool can help identify the server technology being used?

A) Wireshark

B) Nmap

C) WhatWeb

Answer: C

Q2. What tool helps you detect the presence of a reverse proxy?

A) Curl

B) Netcat

C) Nmap

Answer: C

Task 7: Crafting an HTTP Request Smuggle

- Users manually construct raw HTTP requests using netcat or telnet
- Breakdown of each part of a smuggled request:
 - Normal front-end request
 - Embedded malicious backend request (GET /admin)
- Highlights how the smuggled request bypasses inspection.
- MCQ included:

Q1. Which tool is commonly used to send raw HTTP requests to a server?

A) Burp Suite

B) Netcat

C) Firefox

Answer: B

Q2. What is the purpose of sending conflicting Transfer-Encoding and Content-Length headers in a request?

A) To make the request faster

B) To cause discrepancies between proxy and backend server

C) To encrypt the request

Answer: B

Task 8: Exploit Vulnerability

- Users apply their crafted smuggled request to bypass filters and reach /admin
- Upon success, they retrieve the flag
- Encourages testing in multiple scenarios to reinforce learning

- MCQ included:

Q1. What path did the attacker ultimately reach using the smuggled request?

A) /contact

B) /home

C) /admin

Answer: C

Q2. Enter the flag received *Flag: THM{smuggled_into_admin}*

Task 9: Mitigation Techniques

- Discusses how to defend against HTTP Request Smuggling:
 - Updating web servers
 - Disabling conflicting headers
 - Normalizing input at the proxy
 - Using secure default configurations
- Promotes defense-in-depth approach and security hygiene
- MCQ included:

Q1. Which is a common mitigation for HTTP Request Smuggling?

A) Disabling SSL

B) Normalizing headers at the proxy

C) Encrypting database passwords

Answer: B

Q2. How does the server prevent HTTP Request Smuggling?

- A) By rejecting requests with conflicting headers
- B) By redirecting requests to a different server
- C) By blocking IP addresses

Answer: A

Task 10: Cleanup

The objective of this task is to emphasize the importance of proper cleanup after conducting any activity on a system—especially when performing penetration testing, system testing, or any form of exploit demonstration. It teaches responsible behavior in cybersecurity by ensuring that no malicious or sensitive artifacts are left behind that could be misused later. This also reinforces the concept of operational security (OpSec)—minimizing traces and exposure after completing a task.

- MCQ included:

Q1. Why is it important to stop the server and delete logs after completing tasks?

- A) To prevent unauthorized access to the server
- B) To save server resources
- C) To make the system faster

Answer : A

Task 11: Room Summary

- Recaps key learnings:
 - HTTP protocol
 - Proxy-backend interpretation differences
 - Request smuggling theory and exploitation
 - Real-world consequences and mitigations
- Encourages users to research further and practice responsibly
- Final congratulatory message and invitation to connect on TryHackMe forums or Discord
- MCQ included:

Q1. Which vulnerability was the focus of this room?

- A) SQL Injection
- B) HTTP Request Smuggling
- C) Command Injection

Answer: B

Q2. What is the key defense against HTTP Request Smuggling?

- A) Encrypting the request
- B) Validating headers for consistency
- C) Blocking all incoming requests

Answer: B

4. Room Link

Link: - <https://tryhackme.com/jr/testingscL>

5. Reflection

Personal Insights:

Designing *Smuggler's Paradise* was both a technical and creative challenge. It required breaking down a notoriously tricky vulnerability into digestible parts for beginners and advanced learners alike. Crafting analogies to explain backend-header discrepancies gave me a deeper appreciation of how complex and nuanced web communication can be.

Contribution to the Community:

This room adds to the cybersecurity community by offering a structured and approachable way to learn a high-impact vulnerability that's often overlooked. By encouraging users to host their own vulnerable server or use the AttackBox, the room promotes both offensive and defensive thinking — a true full-circle learning experience.

Participants leave the room with not just theoretical knowledge but also the practical ability to spot, exploit, and defend against HTTP Request Smuggling vulnerabilities in real-world applications.